A Comment on CyberChainBench (arXiv:2606.26216): The Headline Difficulty Gradient Is Not an Identified Comparison

Anton Dziatkovskii
Palo Alto AI Research Lab
ORCID 0000-0001-7408-3054 `https://github.com/tonydzi`

October 2026

Abstract

CyberChainBench reports that LLM agents score 37.5% on vulnerability detection, 43.7% on exploitation and 23.4% on patching of real DeFi incidents, and presents this as "a clear difficulty gradient". We audit the paper (v1) against its public release (commit `1cb7410`) without running any model. The release reproduces the 94-case patch subset, and every patch cell of the per-difficulty table is arithmetically consistent with it as a whole number of passes. It does not identify the 200 cases behind 43.7%. The three headline numbers are three differently defined scores on three non-identical case sets. The text and the release disagree with each other in several places: three different best patch scores, two tables that disagree for one configuration, a statement of monotonic degradation that fails in 11 of 30 table rows including the headline configuration, and a blocking condition written as "the attack must revert" while the released scorer also counts a changed return value or log stream as blocked. The leaderboard shipped with the release disagrees with the paper for 6 of 10 configurations. We do not claim the reported numbers are wrong. We claim the released materials do not support reading them as a ranking of task difficulty or as a measure of patch correctness, and we list controls that would help. All checks run from a public script with no API keys and no model calls.

1 Scope

The object of this comment is arXiv:2606.26216v1 [1] and the release `defai-labs/CyberChainBench` at commit `1cb7410` (tag `v1.0`), the only commit of that repository at the time of writing. Every number in Sections 2–5 is quoted from the paper, read from the 541 released case files or the released code, or computed from those by the public script; the boxed illustration in Section 5 comes from separately published raw logs. Tables of the paper were extracted structurally from the arXiv HTML into CSV; the audit script, the extracted tables and a red-first test that corrupts the inputs and checks that each verdict flips are public [2]. No language model, chain or fork was used for Sections 2–5.

2 What reproduces

The release ships 541 case files. The code's patch filter (`fixable`, verified source, at least one historical legitimate transaction) yields exactly 94 cases, the paper's patch subset: 31 easy, 43 medium, 18 hard and 2 labelled `unknown`. With the two unlabelled cases treated as their own group, every patch cell of the paper's Table 4 is arithmetically consistent with a whole number of passes for all ten configurations; the headline 23.4% corresponds to 22 of 94. This checks denominators,

not the underlying model outcomes, which the release does not include. The temporal split (86 incidents dated 2025 or later, 455 earlier) reproduces. The subset counts and the patch arithmetic are consistent with the paper.

3 The headline is three estimands on three sets

Detection is an exact match of vulnerability type and function over 541 cases. Exploitation is the mean of $\min(\text{profit}/\text{reference}, 1)$, a clamped continuous score with iterative attempts, over 200 cases. Patching is the mean of $\mathbf{1}[\text{blocked}] \cdot \mathbf{1}[\text{normal transactions pass}]$, a binary conjunction with iterative attempts, over 94 cases whose overlap with the exploit set is not published. The paper defines each reward carefully; it does not discuss whether the three percentages can be compared, and the abstract reads them as one difficulty scale. Raw percentages of differently defined scores on different case sets do not by themselves identify relative task difficulty.

The 200 exploit cases are not identifiable from the release. When present, the evaluation script reads a generated directory `data/benchmark/harbor_tasks/exploit_subset200` (`src/scripts/2_eval/eval.py:475-477`); that path is excluded by `.gitignore` and not shipped. We found no released code that builds it and no sampling rule in the paper. The exploit headline therefore cannot be independently reproduced from the released materials.

The headline numbers are reported without uncertainty. As a descriptive binomial interval, ignoring oracle error and the selection of the best configuration, 22/94 spans [16.0, 32.9]. As an illustrative sensitivity calculation only, treating the sets as independent and giving the exploit score its maximal variance, exploitation exceeds detection by 6.2 points ($z = 1.52$) and patching falls below exploitation ($z \approx 3.6$); dependence between overlapping sets is not modelled, so these are not tests of a difficulty ordering. Descriptively, exploitation exceeds detection in 7 of 10 configurations and patching is the lowest of the three scores in 9 of 10, Codex × GPT-5.2 being the exception. The ordering of scores is clear for patching; the data do not establish a three-step ordering of task difficulty.

4 Internal inconsistencies

Three best patch scores. Section 4 states "Opus 4.7 leads on patch pass rate (21.3%)". The abstract and the same section state "the best patch pass rate (23.4%)". Table 4 prints 24.5% for Codex × GPT-5.2, and that cell is consistent with its own easy/medium/hard cells. The same Section 4 sentence names Opus 4.7 the realized-profit leader at \$45.5M, while the abstract and Table 5 give \$57.4M to GPT-5.5.

Table 4 against Table 5. Table 4 "Patch All" and Table 5 "Pass %" report the same quantity and agree for nine configurations. For GPT-5.2 they read 24.5 and 17.5. Three Table 5 cells are not $k/94$ for any integer k (GPT-5.2 block 56.1 and pass 17.5, DeepSeek V4 block 54.0); the GPT-5.2 pair is consistent with $k/57$. We report this as an inconsistency and do not assume a typo.

Monotonic degradation. Section 4.2 states "All models show monotonic degradation from easy to hard cases". In Table 4, 11 of 30 task rows are not monotonic, including the headline configuration on both reported tasks: GPT-5.5 exploit 39.8 / 50.5 / 33.2 and patch 19.4 / 32.6 / 11.1 (easy / medium / hard).

The patch reward, as written and as implemented. The paper states the blocking condition four times as "the original attack must revert" (Section 3, the oracle paragraph, the reward definition and the agent prompt); the oracle paragraph adds the alternative "or produce $< 1\%$ of original profit". The tool description and the reward definition say the patch is deployed "via proxy upgrade", while the oracle paragraph says it is injected with `vm.etch`. In the released scorer the attack replay counts as blocked if the call fails *or* its return data changes *or* its log hash changes (`src/eval/tasks/patch/attack_replay.py:227`); the profit test runs only when none of those changed (line 229). The patch is etched at the case's proxy address for the attack replay when one is set (line 260) and at the case's `vulnerable_address` for the legitimate transactions (`src/infra/cli/validate_patch.py:1577`).

Release artefacts. The leaderboard shipped in the same repository differs from Table 4 by more than rounding for 6 of 10 configurations on at least one task (file `site/assets/leaderboard.json`); its builder notes that the Codex rows come from an earlier leaderboard. The paper's filter arithmetic for the patch subset, $153 - 31 - 29$, gives 93, not 94; the release removes 39 and then 20. The site counts the 12 `unknown`-difficulty cases as easy, which is the only way the paper's "42% easy" reproduces. Nine cases carry curator flags marking them disputed or to be skipped (two of them, key compromises, are in the patch subset); no code under `src/` or `site/` reads those flags.

The temporal split as a conditional. The paper gives Opus 4.7 exploit 44.9% before the knowledge cutoff and 38.8% after, and Table 4 gives 42.3% over the exploit set. If these are one weighted mean over the 200 exploit cases, the set must contain about 85 post-cutoff incidents; the corpus contains 86 and a proportional sample would contain about 32. The figures need an explanation: a strong skew of the exploit set toward recent incidents, which would bear on the contamination argument, numbers computed on different populations, or a reporting error. The paper reports no post-cutoff number for patching; only 9 of the 94 patch cases are post-cutoff.

5 What the released scorer tests

Read from the code at `1cb7410`. "Normal operation preserved" is byte equality of the success flag, the return data hash and the hash of the full log stream on one to five historical calls (`validate_patch.py:1330-1332`); a correct fix that changes an amount or an event on that entry point fails it. A legitimate-call replay whose forge output contains the string `MemoryOOG` or `OutOfGas` is marked preserved (lines 1608–1612), and the reported number of checked calls is the number of result rows (line 1638), not the number of replays that printed the check marker. On the verified-source path, recompilation hard-codes `optimizer: enabled, runs: 200` and passes no EVM version (`src/infra/solidity/legacy.py:226`); this function is called by the scoring path (`src/eval/tasks/patch/evaluator.py:741`). In the path we traced, the explorer's verified compiler version is fetched (`src/infra/chain/metadata.py:838`) but only optimizer and EVM settings are copied from it (`validate_patch.py:1373-1382`). These properties push in both directions. Byte equality can reject correct fixes. Separately, any-change blocking can let a patch through the blocking check without stopping the attack, and the out-of-gas rule can let a patch through the preservation check without the replay being verified; full credit requires passing both. We found no calibration of the oracle against no-op patches or reference fixes in the paper or the release.

Illustration, not a rate. We ran the released `validate_patch` command-line tool at `1cb7410`, without code changes, on forks of historical Ethereum state for eight cases with three fixed inputs each and no model. Five cases gave the same score to all three inputs and are uninformative on this setup. On bench-391, a patch that only emits an additional event in the labelled function scored 1.0 while the tool reported identical attacker profit before and after the patch; the attack counted as blocked because the log hash changed. This is the any-change rule of Section 4 observed on a real case. It was obtained through the command-line path; equivalence with the scoring path used for the paper's tables was not tested. Results for recompiled unchanged sources are not interpreted here, because recompilation is not a null intervention: the on-chain-bytecode control has not been run. Raw logs are in the repository [2].

6 What this does and does not show

It does not show that 23.4% or 43.7% are wrong, nor the direction or size of any bias. It shows that the released materials do not identify the exploit set, that the three headline numbers are not one scale, that the paper's text, tables and shipped leaderboard disagree in the places listed, and that the implemented blocking rule is broader than the one described. These limits are of the same family as the plausible-but-incorrect patches long documented in automated program repair [3, 4] and recently measured for agent benchmarks in other domains [5, 6]; what is specific here is the on-chain replay oracle.

Controls that would help interpret the patch number as patch correctness under this oracle: for each case, a hosting control that etches the on-chain runtime and checks that the historical attack still pays; on hosted cases only, adversarial negative controls (`revert()`, emit-only) with their expected component outcomes stated in advance, and positive controls with independently checked reference fixes; and a rescoring of the same traces with a predicate that requires attacker profit to fall and does not count out-of-gas as preservation. Any statement about relative task difficulty would additionally need the exploit case list, same-case comparisons and matched attempt budgets.

Prior work and corrections

Before this comment, a contributor opened an issue and seven pull requests on the release addressing replay accounting, null proxy addresses, prompt and scorer alignment and build settings [7]; we credit that work and verified in code only the points cited above. An earlier public version of our audit overstated several claims (that the headline numbers were confirmed, that forks had been run, the composition effect, and the extent of external review); the corrections are listed in the repository changelog [2].

Use of AI tools

The audit scripts, the reading of the release and drafts of this text were produced with AI assistance (Anthropic Claude); adversarial reviews of intermediate conclusions were run with OpenAI Codex, xAI Grok, Google Gemini and Z.ai GLM. One of those reviews refuted a stronger claim in an earlier version, and the refutation is published with the prompts. The author takes responsibility for every claim. Sections 2–5 are reproduced by `make` in the repository; the boxed illustration is reproduced from `calibration/` with an archive RPC endpoint.

Reproduction

```
git clone https://github.com/tonydzi/cyberchainbench-audit
cd cyberchainbench-audit
make        # clones the release at 1cb7410, runs 16 checks
make red    # corrupts the inputs and shows each verdict flip
```

References

[1] J. Huang, F. Jiang, R. Poovendran, Z. Lin. *CyberChainBench: Can AI Agents Secure Smart Contracts Against Real-World On-Chain Vulnerabilities?* arXiv:2606.26216v1, 2026. Release: `https://github.com/defai-labs/CyberChainBench`, commit 1cb7410.

[2] A. Dziatkovskii. *cyberchainbench-audit.* `https://github.com/tonydzi/cyberchainbench-audit`, 2026.

[3] Z. Qi, F. Long, S. Achour, M. Rinard. An analysis of patch plausibility and correctness for generate-and-validate patch generation systems. In *ISSTA*, 2015.

[4] E. K. Smith, E. T. Barr, C. Le Goues, Y. Brun. Is the cure worse than the disease? Overfitting in automated program repair. In *ESEC/FSE*, 2015.

[5] C. Shen, J. Li, A. Mahajan, J. S. Tian, Y. Kwon, Y. Chen. *PatchBench: Evaluating AI Agents for Vulnerability Patching.* arXiv:2609.04075, 2026.

[6] A. Tuzikov. *RoboBugBench: Benchmarking Automated Repair of Real Robot Software Under an Audited Behavioral Oracle.* arXiv:2610.16369, 2026.

[7] shane-moore. Issue #8 and pull requests #1–#7 on `defai-labs/CyberChainBench`, October 2026.